grounding_agent — Evaluation Framework (τ^3-bench)

Vineet Singh

May 13, 2026

CONTENTS

grounding_agent — presentation

- In one sentence
- What we built
- Seven-category failure taxonomy
- Generated contract
- Runner adapter
- Structured event log
- 125 tests
- How we built it — the journey
- Day 1 (initial build, against τ-bench)
- Day 2 (initial eval, against τ-bench)
- Forensics iterations 1–3 (still on τ-bench)
- Iteration 4 — the meta-discovery
- Forensics iterations 1–3 on τ³-bench (final)
- Key numbers (τ³-bench iter-3 final state)
- Variant overview
- Before / after the τ³-bench migration
- Per-dimension pass rate (iter-3 final)
- Judge cost decomposition (from event log)

What the framework caught that vibes-eval would not

What's missing / known limitations

Way forward

Cost + ops at production scale

What we learned (the meta-points)

Where everything lives

grounding_agent — presentation

> Submission for Meraki Labs Founding AI Engineer Work Trial, PS4 — *Evaluation Framework from Scratch*. Read time ≈ 5 minutes.

In one sentence

A multi-dimensional evaluation framework for tool-calling LLM agents, evaluated against **τ³-bench** (the current benchmark from Sierra Research — the original τ-bench is obsolete) on the airline customer- support agent, where the **disagreements between auto-eval and the benchmark's programmatic reward are the result** and where structured forensic iteration turned the framework's own weaknesses into measured improvements.

What we built

SEVEN-CATEGORY FAILURE TAXONOMY

Frozen taxonomy of structural failure modes for tool-using agents. Each category grounded in a load-bearing policy clause.

Category	Judge kind	What it catches
`policy_compliance`	semantic	Business rules ignored or applied wrong
`confirmation_discipline`	**deterministic**	Mutating tool call without preceding user "yes"
`information_grounding`	semantic	Facts not in tool outputs / user messages
`scope_adherence`	semantic	Mis-applied transfer-to-human decision
`tool_sequence_correctness`	**deterministic**	Mutation without prerequisite read
`tool_argument_correctness`	**deterministic**	Tool returned `Error:` (now reads τ³-bench's `ToolMessage.error` flag)
`task_completion`	observed via reward	End-to-end goal achievement

3 semantic + 3 deterministic + 1 reward-observed = **7 dimensions, 6 judges.**

GENERATED CONTRACT

One LLM call against `vendor/tau_bench_airline/policy.md` produces `data/contract.json` (15 clauses across obligations + forbidden behaviors + tool_sequences, each tagged to a taxonomy category). Validator gates save and load. **No hand-curated clause text**.

RUNNER ADAPTER

`grounding_agent/runner.py` ports between **two backends**:

- Originally: `sierra-research/tau-bench` (the old one).
- Now: `sierra-research/tau2-bench` (which ships τ³-bench).

Adapter flattens pydantic Message types into OpenAI shape, maps τ³-bench's first-class `TerminationReason` enum to our termination kinds, reads `ToolMessage.error: bool` for the argument-correctness check. **The rest of the framework (taxonomy, contract, judges, evaluator, compare, eventlog) did not need to change** — the framework is portable.

STRUCTURED EVENT LOG

JSON-Lines at `results/logs/<run_id>/<variant>.jsonl`. Per-task and per-judge events with timing and verdicts. Replayable.

125 TESTS

All passing. Test fixtures use the actual τ³-bench message shapes and exercise the adapter in isolation.

How we built it — the journey

DAY 1 (INITIAL BUILD, AGAINST T-BENCH)

Scaffold → taxonomy → contract → judges → runner → evaluator → smoke test on 2 tasks. Six modules, all under 300 lines.

DAY 2 (INITIAL EVAL, AGAINST T-BENCH)

Full 20×2 evaluation. First forensics pass surfaced six load-bearing findings.

FORENSICS ITERATIONS 1–3 (STILL ON T-BENCH)

Bucketed four issue classes (misclassified judge, wrong judge input shape, missing dimension, reporting gaps); fixed them; re-ran; re-mined; iterated on contract mistagging. Reward stayed flat at ~10% across variants. v2 looked WORSE than v0 (5% vs 16%).

ITERATION 4 — THE META-DISCOVERY

A web check revealed **τ-bench is obsolete**. Sierra released τ²-bench (2025) and τ³-bench (March 2026). τ³-bench **fixed 27 airline tasks** — incorrect expected actions, ambiguous user instructions, impossible constraints, missing fallbacks, policy loophole closures. Per Sierra's own blog: airline pass^1 scores improved **+14 to +20 points** after the fixes.

Implication: **iterations 1–3 partially measured ground-truth bugs, not agent failures.** Migrated the framework to τ³-bench. Re-ran all three iterations.

FORENSICS ITERATIONS 1–3 ON T³-BENCH (FINAL)

- Iter-1: surfaced two new issues (the τ³-bench contract mistagged the multi-tool-call clause as scope_adherence; reward_kind didn't understand τ³-bench's richer `RewardInfo` shape).
- Iter-2: hand-patched the contract; updated `reward_kind()`; added missing `confirmation_discipline` and `tool_argument_correctness` clauses. Rejudge under new contract. tool_sequence_correctness moved 65% → 75% on v0.
- Iter-3: refined `scope_adherence` clause text. information_grounding unstuck (65% → 80% on v0) but `scope_adherence` returned to 0% — the LLM judge cannot reliably decide whether a transfer was scope- appropriate.

Key numbers (τ³-bench iter-3 final state)

VARIANT OVERVIEW

metric	v0 (wiki as-is)	v2 (discipline preamble)
τ³-bench reward (all)	**30%**	**35%**
reward (train)	50%	20%
reward (held-out)	10%	**50%**
avg messages / run	22.7	24.2
total cost	$0.09	$0.13
termination: completed	7	8
termination: transfer	6	6
termination: max_steps	7	6
tool errors observed	6	6

BEFORE / AFTER THE T³-BENCH MIGRATION

metric	τ-bench iter-3	τ³-bench iter-3	net
v0 reward (all)	10%	**30%**	**+20pp**
v2 reward (all)	10%	**35%**	**+25pp**
v2 reward (held-out)	10%	**50%**	**+40pp**
v2 vs v0 (held-out)	-10pp	**+40pp**	**sign flipped**

The agent did not change. The benchmark did. **About half of the agent's apparent failure on the old τ-bench was broken ground truth**, confirming Sierra's claim.

PER-DIMENSION PASS RATE (ITER-3 FINAL)

dimension	v0	v2
`confirmation_discipline`	70%	70%
`information_grounding`	80%	70%
`policy_compliance`	25%	15%
`scope_adherence`	**0%** ⚠	0% ⚠
`tool_sequence_correctness`	75%	85%
`tool_argument_correctness`	75%	80%

JUDGE COST DECOMPOSITION (FROM EVENT LOG)

dimension	mean duration	kind
`policy_compliance`	2 900 ms	semantic
`scope_adherence`	2 700 ms	semantic
`information_grounding`	1 900 ms	semantic
All 3 deterministic judges	**< 1 ms each**	deterministic

What the framework caught that vibes-eval would not

1. **The benchmark itself was broken** (in the original τ-bench run). The framework's structured, per-dimension verdicts made it natural to investigate "why does v2 produce more arithmetic tool errors yet sometimes have higher reward?" — which led to the τ³-bench discovery.
2. **v2 generalises better than v0 on held-out** (50% vs 10% on τ³-bench). On the broken τ-bench this was invisible. Multi- dimensional eval surfaces variant-vs-variant deltas that single- score eval would average out.
3. `confirmation_discipline` **LLM judge was strictly worse than a 10-line Python heuristic.** Reclassified as deterministic; pass- rate went from 0% → 70%. Same fix held under τ³-bench.

4. **Arithmetic dominates agent failures.** Added a new dimension that reads τ³-bench's `ToolMessage.error: bool` flag directly. The flag is a cleaner signal than our prior grep-for-`Error:` heuristic.
5. `scope_adherence` **is the dimension where the LLM-judge approach hits its limit.** Three iterations of clause refinement couldn't get the judge to reliably decide "was the user's request in-scope?". Honest finding worth documenting.

What's missing / known limitations

gap	why	fix
`scope_adherence` 0% pass-rate persists	LLM judge can't reliably evaluate scope from a single clause	Multi-shot panel, gpt-4o, or drop the dimension
Argument-CHOICE correctness (right flight, right cabin, right split)	Bucket C catches *invalid* args via Error returns; not *suboptimal* args	New semantic judge re-deriving expected args from tool returns + user constraints
Communicate-checks dimension (τ³-bench's nl_assertions / communicate_checks)	Framework doesn't read these	Surface a `response_quality` dimension that reads `RewardInfo.nl_assertions`
Single trial per task (n=20)	Cost discipline	Multi-trial at higher N
Synchronous judges	Sufficient at n=40	`evaluate_trajectory` is pure; async-ify in ~30 min
One judge model (gpt-4o-mini)	Cost	Tier: deterministic on 100%, gpt-4o-mini on sample, gpt-4o on flagged

---

Way forward

1. **Surface** `RewardInfo` **decomposition** (db_check + env_assertions
    - action_checks + nl_assertions + communicate_checks). Each is a first-class scoring dimension in τ³-bench. The framework currently ignores them; adding visibility would catch what we're missing.

2. `tool_argument_choice_correctness` semantic judge. Close the coverage gap that produces most remaining FPs.
3. **Drop or reframe** `scope_adherence`**.** Three iterations failed to stabilise it; accept the dimension as inherently noisy or remove.
4. **Multi-trial at N ≥ 50** for tighter v0/v2 deltas.
5. **Tier production judges by stakes** — deterministic 100%, semantic sample, gpt-4o arbitration.
6. **One human-in-the-loop tag-review step** after each contract generation. The forensic iterations 2+3 essentially WERE that review; productizing it would prevent the mistag-of-the-week.
7. **Multi-agent comparison.** The framework evaluates one agent right now; the plumbing for "evaluate any tool-using agent" is one indirection.

Cost + ops at production scale

- **Per-trajectory cost** (τ³-bench, gpt-4o-mini end-to-end): ~~$0.012 (agent + user-sim + 3 semantic judges + 3 free deterministic). At 100k/day = **~~$1.2k/day**.
- **Reducing cost without losing signal**:
  - Deterministic judges (free, < 1 ms each) on 100% of traffic.
  - Sample semantic judges to 1–5%.
  - gpt-4o for arbitration on flagged trajectories only.
- **Reliability**: errors recorded per-task, don't crash runs. Idempotent caching; per-task crash safety. Validator infrastructure prevents malformed contracts from being silently loaded.
- **Observability**: every run produces a JSON-Lines event log + per- task results JSON + a rendered comparison markdown. All three are mineable.

What we learned (the meta-points)

1. **Multi-dimensional eval's value is in the disagreements with ground truth, not the agreements.** Including disagreements that exposed the ground truth itself was broken.
2. **LLM judges and deterministic judges are not interchangeable.** Confirmation, tool-call ordering, and tool-error detection are mechanically observable; LLMs were strictly worse. Reserve LLM judges for genuinely subjective dimensions (policy compliance, information grounding).

3. `scope_adherence` **is where the LLM-judge approach breaks.** Deciding "was the user's request in-scope" needs holding the whole policy + tool catalog in working memory. gpt-4o-mini won't do it reliably.
4. **Forensics-driven iteration converges fast** (1–2 iterations produce major shifts; subsequent iterations are 1-cell shifts). This held under both τ-bench AND τ³-bench. Plateau is real.
5. **Benchmarks themselves can be broken.** A framework whose value depends on a benchmark's ground truth needs **independent verification** of that ground truth — at least once. The web check that revealed τ³-bench is the kind of cheap due-diligence step that should be in every eval-framework workflow.
6. **Portability earns its keep.** Migrating from τ-bench to τ³-bench required rewriting one module (`runner.py`) + patching one helper (`compare.reward_kind`). The taxonomy, judges, evaluator, eventlog were untouched. That's the framework being agnostic to the agent-under-test, by design.

Where everything lives

artifact	path
Code	`grounding_agent/` (7 modules, all ≤ 300 lines)
Tests	`tests/` (125 passing)
Contract	`data/contract.json` (+ `.tau1.json`, `.tau3.iter1.json` backups)
Vendored policy	`vendor/tau_bench_airline/policy.md` (+ `.tau1.md` backup)
Task split	`data/tasks.json` (τ³-bench train + held-out)
Event logs	`results/logs/<run_id>/<variant>.jsonl`
Forensics (τ-bench)	`forensics.md`, `forensics_v2.md`, `forensics_v3.md`
Forensics (τ³-bench)	`forensics_tau3.md`, `forensics_tau3_v3.md`
Migration doc	`results/tau1_vs_tau3.md`
Comparison reports	`results/comparison.md` (+ many backups)
Per-iteration results	`results/{v0,v2}_results{.tau1.iter3,.tau3.iter1,.tau3.iter2,}.json`
Code reviews	`code_review/` (per-chunk dated reviews)
Session log	`knowledge.md`
Error log	`errors.md`
Project brief	`BRIEF.md`
Long-form writeup	`WRITEUP.md`
This file	`PRESENTATION.md`

artifact	path
Repo entry point	`README.md`